

DAA Lab - Experiment: Naïve vs Rabin–Karp

String Matching Algorithms

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement and compare the Naïve string matching and Rabin–Karp (rolling hash) algorithms.
- Analyze time complexity, collisions, and practical performance.
- Demonstrate pattern searching on sample text.

- **Naïve:** Check pattern at every possible shift. Worst-case time $O(mn)$ for text length n and pattern length m .
- **Rabin–Karp:** Compute rolling hash of length- m windows in text and compare to pattern hash; on hash match, verify characters to avoid false positive. Average time $O(n + m)$, worst-case $O(mn)$ with many collisions.
- Rolling hash: common choice is modular arithmetic with a base (e.g. 256) and prime modulus.

Pseudocode: Naïve

- ① For $s = 0$ to $n-m$:
 - Compare $P[0..m-1]$ with $T[s..s+m-1]$
 - If all match, report shift s

Pseudocode: Rabin–Karp

- ① Precompute hash of pattern and first window of text.
- ② For $s = 0$ to $n-m$:
 - If $hash_P == hash_{Twindow}$ then verify characters; if match report s
 - Slide window: update $hash_{Twindow}$ using rolling formula

C Code - Naïve (Part 1)

```
#include <stdio.h>
#include <string.h>

void naive_search(const char *text, const char *pat) {
    int n = strlen(text);
    int m = strlen(pat);
    for (int s = 0; s <= n - m; s++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[s + j] != pat[j]) break;
        }
        if (j == m) printf("Pattern found at index %d\n", s);
    }
}
```

C Code - Rabin-Karp (Part 1)

```
#include <stdlib.h>

#define D 256 /* number of characters */
#define Q 101 /* a prime modulus for hashing */

int mod_pow(int base, int exp) {
    long long r = 1, b = base;
    while (exp > 0) {
        if (exp & 1) r = (r * b) % Q;
        b = (b * b) % Q;
        exp >>= 1;
    }
    return (int)r;
}
```

C Code - Rabin-Karp (Part 2)

```
void rabin_karp(const char *text, const char *pat) {
    int n = strlen(text), m = strlen(pat);
    if (m > n) return;

    int h = 1; //  $h = (D^{(m-1)}) \% Q$ 
    for (int i = 0; i < m - 1; i++) h = (h * D) % Q;

    int p = 0, t = 0; // hash values
    for (int i = 0; i < m; i++) {
        p = (D * p + pat[i]) % Q;
        t = (D * t + text[i]) % Q;
    }
}
```


C Code - Rabin-Karp (Part 3)

```
for (int s = 0; s <= n - m; s++) {
    if (p == t) { // possible match, verify
        int match = 1;
        for (int i = 0; i < m; i++) {
            if (text[s + i] != pat[i]) { match = 0; break; }
        }
        if (match) printf("Pattern found at index %d\n", s);
    }
    if (s < n - m) {
        // remove leading, add trailing
        t = (D * (t - text[s] * h) + text[s + m]) % Q;
        if (t < 0) t += Q;
    }
}
```

C Code - Main (Part 1)

```
int main() {
    char text[1000], pat[100];
    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    // remove newline if present
    size_t ln = strlen(text);
    if (ln && text[ln-1] == '\n') text[ln-1] = '\0';

    printf("Enter pattern: ");
    fgets(pat, sizeof(pat), stdin);
    ln = strlen(pat);
    if (ln && pat[ln-1] == '\n') pat[ln-1] = '\0';
```

C Code - Main (Part 2)

```
printf("\nNaive Search Results:\n");  
naive_search(text, pat);  
  
printf("\nRabin-Karp Search Results:\n");  
rabin_karp(text, pat);  
  
return 0;  
}
```

Sample Input / Output

Input:

Text: "ABABDABACDABABCABAB"

Pattern: "ABABCABAB"

Output (both):

Pattern found at index 10

Complexity Analysis

- Naïve: Worst-case $O(mn)$, best-case $O(n)$ (no matches early).
- Rabin–Karp: Average $O(n + m)$ with good hash; worst-case $O(mn)$ due to hash collisions and verification.
- Space: both use $O(1)$ extra (except input/storage).
- Practical note: choose modulus Q and base D to minimize collisions.

Observation Table (example)

Text length n	Pattern length m	Naïve (ms)	Rabin-Karp (ms)
1000	5	0.2	0.1
1000	50	1.8	0.6
5000	50	9.2	2.1

Comparison

- Naïve is simple, predictable, but slow on repetitive texts.
- Rabin–Karp excels when searching multiple patterns (hashing helps).
- Rabin–Karp requires careful choice of hash (base, modulus) to avoid collisions.
- Use Rabin–Karp for early filtering; always verify character-by-character on hash match.

Viva Questions

- 1 Explain how the rolling hash is computed and updated.
- 2 Why do we verify characters after a hash match?
- 3 What causes Rabin–Karp to degrade to $O(mn)$?
- 4 How to choose modulus Q and base D ?

Conclusion

- Both algorithms are useful: Naïve for simplicity, Rabin–Karp for efficiency with hashing.
- Rabin–Karp is especially useful for multiple-pattern search and initial filtering.
- Always account for collisions; verify matches to ensure correctness.

References

- Cormen et al., *Introduction to Algorithms*.
- Dan Gusfield, *Algorithms on Strings, Trees and Sequences*.